The Journal of Open Source Software

NPAP: Network Partitioning and Aggregation Package for Python

Marco Anarmo [1,2], **Benjamin Stöckl** [1,2], **Yannick Werner** [1,2], **and Sonja Wogrin** [1,2]

1 Institute of Electricity Economics and Energy Innovation (IEE), Graz University of Technology, Inffeldgasse 18, Graz, Austria **2** Research Center ENERGETIC, Graz University of Technology, Rechbauerstraße 12, Graz, Austria

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Vincent Knight

Reviewers:

- @fneum
- @FBumann
- @trintlermint
- @amcandio
- @abhishektiwari

Submitted: 10 April 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

NPAP (Network Partitioning and Aggregation Package) is an open-source Python library for reducing the spatial complexity of network graphs. Built on NetworkX (Hagberg et al., 2008), it provides an accessible standalone package designed to be readily integrated with other software and frameworks. Instead of treating the spatial reduction process as a single action, NPAP explicitly splits it into two distinct steps: partitioning, which assigns vertices (nodes) to groups (clusters), and aggregation, which reduces the network based on a given assignment. NPAP's strategy pattern architecture allows users to employ and register custom partitioning and aggregation strategies seamlessly without modifying the core code. Currently, NPAP provides 18 partitioning strategies and two pre-defined aggregation profiles. Although initially developed with a focus on power systems, its architecture is general-purpose and applicable to any network graph.

Statement of need

Real-world electricity grids have grown significantly in size and complexity, potentially spanning many thousands of nodes and edges. Consequently, energy system optimization models representing those networks have become computationally intractable and challenging to solve. To regain tractability and computational efficiency, modelers frequently apply temporal and spatial aggregation techniques (Kotzur et al., 2021). Today, temporal complexity reduction, known as time series aggregation, is a well-established field (Hoffmann et al., 2020; Teichgraeber & Brandt, 2022; Wogrin, 2023). Tools like tsam (Time Series Aggregation Module) (Hoffmann et al., 2022) have consolidated multiple temporal aggregation algorithms into a single, reusable Python library that has become a standard component in many energy system optimization modeling frameworks. Before tsam, modelers implemented these methods individually and ad hoc for each project, challenging reusability and comparability.

This is exactly where the research on spatial complexity reduction is at: While there is some research on network partitioning (Frysztacki et al., 2022) and aggregation (Colonetti, 2025) methods available, there exists no standalone tool that brings all methods together and is easy to use, extend, and does not rely on framework-specific data structures. NPAP fills this gap as a standalone package that works with any NetworkX (Hagberg et al., 2008) graph. The target audience includes power systems researchers, energy modelers, network analysts, and more broadly, anyone interested in reducing the spatial complexity of a graph.

DRAFT

State of the field

Within the energy system optimization community, existing implementations of network partitioning and aggregation methods are usually tightly coupled to the respective frameworks' internal data structure and cannot be reused independently or by other tools and frameworks. The built-in spatial clustering module of PyPSA (Brown et al., 2018), for example, provides busmap-based spatial aggregation using methods such as k-means and hierarchical clustering from scikit-learn (Pedregosa et al., 2011), and modularity-based community detection from NetworkX (Hagberg et al., 2008), but implements the bus weightings and the aggregation of component attributes specifically for PyPSA's `Network` object. As another example, ETHOS.FINE (Klütz et al., 2025) implements spatial reduction through string-, distance-, and parameter-based partitioning modes (Patil et al., 2022), which are strictly tied to the framework's internal region-based data model. Researchers working with other frameworks or custom network representations cannot leverage these implementations without significant adaptation to the core code structure.

The analogy with temporal complexity reduction and the tsam package (Hoffmann et al., 2022) is instructive. NPAP extends this paradigm to the spatial dimension. Its unique contributions include:

- Standalone Architecture: A pip-installable library decoupled from any specific energy system modeling framework.
- Extensibility: A strategy pattern architecture that allows new functionalities to be added without modifying core code.
- Pipeline Control: An explicit conceptual separation between partitioning and aggregation, enabling rigorous control over each stage of the reduction pipeline.
- Advanced Physical Constraints: Built-in support for voltage-aware and AC-island-aware partitioning for real-world electricity networks.

Pipeline architecture

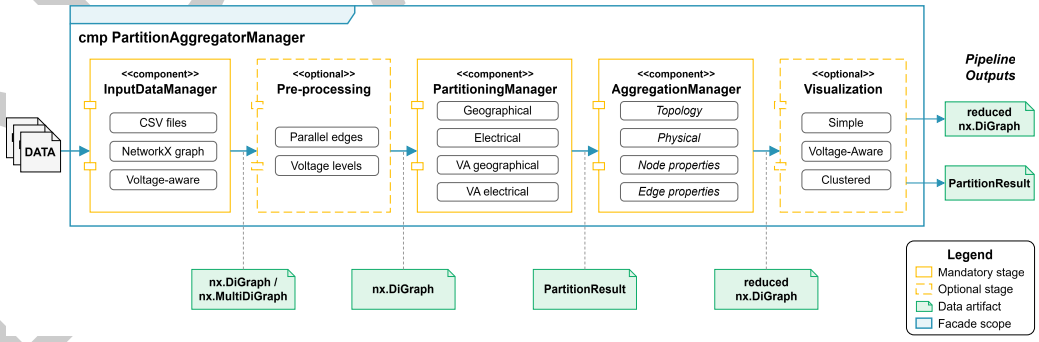


Figure 1: NPAP pipeline architecture.

The full NPAP pipeline is shown in Figure 1. Initially, NPAP performs two stages, preparing the network graph. In the first stage, data is loaded and a NetworkX graph — used for the reduction process — is created and validated. In the second stage, NPAP provides optional pre-processing steps that prepare the network graph for the reduction process. The network can be illustrated throughout the pipeline using the visualization component.

A key design decision in NPAP is the explicit separation of the network reduction process into two stages:

1. Partitioning: A partitioning strategy maps each node in the original network to a node in the aggregated network. This step only determines group "membership"; it does not

DRAFT

modify the graph itself.
2. Aggregation: An aggregation strategy reduces the network topology based on a given partitioning result (mapping) by aggregating nodes, edges, and their associated properties.

This separation was identified early in the design process as a fundamental distinction. It gives users fine-grained control: they can swap partitioning algorithms independently of the aggregation method, apply different aggregation strategies to the same partitioning result, compose custom pipelines that combine strategies from different domains, or simply use any of them in isolation. The next section explains the software design in more detail.

Software design

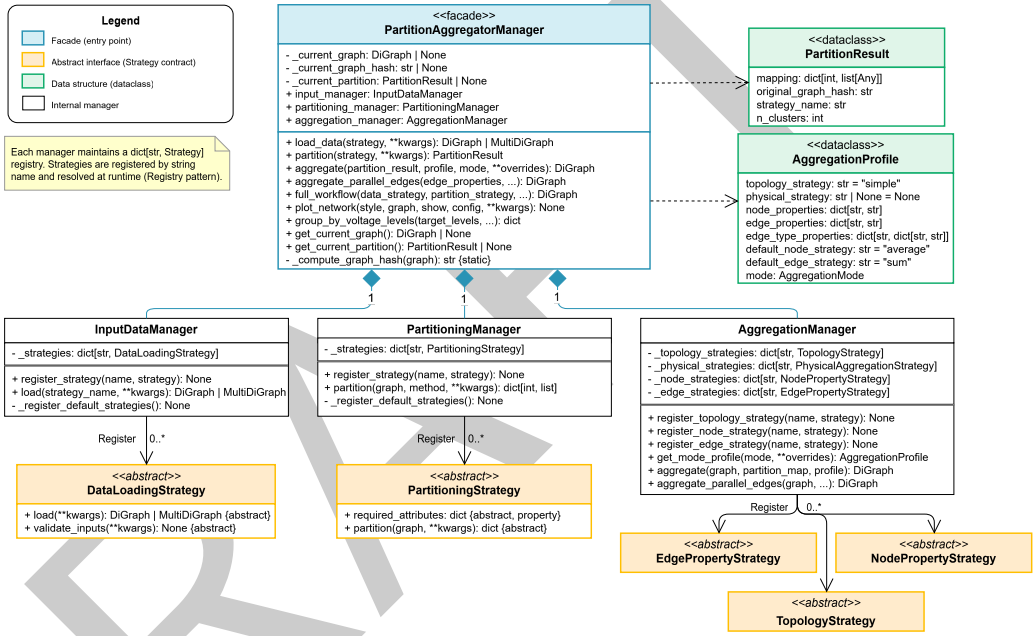


Figure 2: NPAP general class diagram.

A broad overview of NPAP's design is shown in Figure 2. The whole workflow is orchestrated and accessed by the `PartitionAggregatorManager`. NPAP follows a strategy pattern with five categories — data loading, partitioning, topology aggregation, physical aggregation, and property aggregation — all orchestrated by three manager classes, shown at the bottom of the Figure. New strategies inherit from abstract base classes and register with their respective managers, enabling users to seamlessly add custom strategies without modifying core code. We chose this strategy pattern over a monolithic, flag-driven implementation to keep methods self-contained and make contributing new strategies straightforward, accepting additional boilerplate and indirection through the manager classes in exchange for better structure and maintainability. To further lower the entry barrier, the whole library is built on well-established Python packages, such as NetworkX (Hagberg et al., 2008) for graph representation, pandas (McKinney, 2010) for tabular data handling, NumPy (Harris et al., 2020), SciPy (Virtanen et al., 2020), and scikit-learn (Pedregosa et al., 2011) for numerical computation, and Plotly (Plotly Technologies Inc., 2015) for interactive map-based visualization of results. In the following sections, we introduce the loading and pre-processing of input data, partitioning, and the aggregation processes in more detail.

DRAFT

Data loading and pre-processing

As NPAP is designed to work with NetworkX graphs, they can be directly passed as input data, facilitating an easy integration into other frameworks or software. Besides, NPAP currently supports importing data from CSV files with two main strategies. The first one works with general node and edge data targeting generic graph structures. The second one is a domain-dependent strategy focused on power grids and includes buses, lines, transformers, converters, and DC links. User-specific data loading strategies can be registered easily, as shown in Figure 2. Afterward, optional pre-processing steps are carried out through the `PartitionAggregatorManager`, such as the aggregation of parallel edges. For power grids, voltage-level grouping separates the network into independent sub-graphs per voltage level, enabling voltage-aware partitioning strategies.

Partitioning strategies

NPAP currently provides four families of partitioning strategies combining geographical and electrical node distance with the option of partitioning voltage levels independently (voltage-awareness). Across these families, NPAP offers 18 partitioning strategies based on five well-established clustering algorithms: k-means (Lloyd, 1982), k-medoids (Schubert & Lenssen, 2022; Schubert & Rousseeuw, 2021), DBSCAN (Ester et al., 1996), HDBSCAN (Campello et al., 2013; McInnes et al., 2017), and hierarchical clustering (Ward, 1963). Geographical distance captures latitudes and longitudes and is suitable for any geo-referenced network. Electrical distance computes Power Transfer Distribution Factors (PTDFs) (Wood et al., 2013) to capture the electrical behavior of a power grid rather than its geographical topology.

In power grids, NPAP automatically detects alternating current (AC) islands linked solely by DC interconnections and partitions them independently. This, and the voltage-aware partitioning, is achieved by setting the distance matrix entries of nodes in different AC islands and voltage levels, respectively, to infinity. Both approaches are algorithm-agnostic, i.e., they work with any distance-based partitioning method without requiring modifications to the algorithm itself. The partitioning outcome, along with other metadata, is then stored in the `PartitionResult`.

Three-tier aggregation strategy pipeline

The aggregation process is decomposed into three sequential steps. The first (topology) tier builds a new network graph based on the mapping result of the partitioning process by creating a representative node for each cluster and adjusting the edges between them accordingly. Afterward, to cover use cases we do not foresee and usability outside the power system domain, we have included an optional second (domain-specific) tier. There, users can modify physical properties of the network graph, e.g., adding new edges that did not exist in the original graph or explicitly modifying certain properties, such as line reactances. In the third (property aggregation) tier, user-specified functions, e.g., sum, average, or equivalent reactances, are then applied to the remaining node and edge properties. These property aggregation strategies can be configured by the user in detail through the aggregation profiles shown in Figure 2. At the moment, NPAP provides two pre-defined aggregation profiles.

Research impact statement

NPAP is designed as a standalone, pip-installable open-source Python package, released under the MIT license. The library is fully documented on ReadTheDocs and includes an automated test suite with continuous integration across Python 3.10 to 3.14. It has been tested on small- and large-scale networks with up to around 7,000 nodes.

One potential application of NPAP is within energy system modeling frameworks. For illustration purposes, we have opened a pull request to the well-established PyPSA (Brown et al., 2018) framework that adds NPAP as an optional alternative to PyPSA's native spatial clustering

DRAFT

backend.[1] The integration is self-contained: it converts PyPSA's network object into a NetworkX graph, applies NPAP's partitioning and aggregation strategies, and passes the result back to PyPSA's existing clustering and mapping pipeline. This showcases how NPAP's framework-agnostic design can be leveraged by existing tools without changing their original data model.

We further utilized this implementation to demonstrate NPAP's usability and scalability by applying it to the full pan-European transmission network (Xiong et al., 2025, 2026), comprising around 6,800 nodes and 17,500 edges, within the PyPSA-Eur workflow (Hörsch et al., 2018). In Stöckl et al. (2026), we use k-medoids clustering (Schubert & Rousseeuw, 2021) with geographical (haversine) distances to compare voltage-unaware and voltage-aware partitioning and aggregation, and analyze the resulting infrastructure investments for power lines on different voltage levels and transformers in a transmission expansion planning problem solved with PyPSA (Brown et al., 2018).

Active development directions include the extension of the available partitioning strategies, e.g., the Adjacent Node Agglomerative Clustering algorithm (Stöckl et al., 2025), and aggregation strategies, e.g., based on PTDFs (Fortenbacher et al., 2018), as well as the extension to other physical infrastructures, such as hydrogen networks, which require distinct strategies. Future extensions include, among others, a filtering module that allows selecting subsets of the network, e.g., a country or region, as an additional pre-processing step before partitioning.

AI Usage Disclosure

Several artificial intelligence (AI) tools were used during the development of NPAP and this manuscript. Google Gemini (Pro) was used for non-code-related tasks, such as learning about power systems. Anthropic Claude (Sonnet 4.5, Opus 4.5, 4.6, and 5.5), accessed through both the desktop application and the Claude Code terminal interface, was used for code-related activities, including software design brainstorming, implementation support, test coverage, or writing the documentation. It was also used for consistency and reference checks during the manuscript preparation. Grammarly was used for grammar and spell checking. No other code-generation tools (such as GitHub Copilot, Codex, or Cursor) were used. All AI-generated output was carefully reviewed, tested, and frequently modified before inclusion. The authors take full responsibility for the code and the manuscript.

Acknowledgements

The work done on the package by the contributors of the Institute of Electricity Economics and Energy Innovation (IEE) at Graz University of Technology was funded by the European Union (ERC, NetZero-Opt, 101116212). Views and opinions expressed are, however, those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

References

Brown, T., Hörsch, J., & Schlachtberger, D. (2018). PyPSA: Python for power system analysis. *Journal of Open Research Software*, *6*(1), 4. https://doi.org/10.5334/jors.188

Campello, R. J. G. B., Moulavi, D., & Sander, J. (2013). Density-based clustering based on hierarchical density estimates. *Advances in Knowledge Discovery and Data Mining*, 160–172. https://doi.org/10.1007/978-3-642-37456-2_14

[1] https://github.com/PyPSA/PyPSA/pull/1568

DRAFT

Colonetti, B. (2025). Ward reduction in unit-commitment problems. *Electric Power Systems Research*, *240*, 111271. https://doi.org/10.1016/j.epsr.2024.111271

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, 226–231. https://cdn.aaai.org/KDD/1996/KDD96-037.pdf

Fortenbacher, P., Demiray, T., & Schaffner, C. (2018). Transmission network reduction method using nonlinear optimization. *2018 Power Systems Computation Conference (PSCC)*, 1–7. https://doi.org/10.23919/pscc.2018.8442974

Frysztacki, M. M., Recht, G., & Brown, T. (2022). A comparison of clustering methods for the spatial reduction of renewable electricity optimisation models of Europe. *Energy Informatics*, *5*(1), 4. https://doi.org/10.1186/s42162-022-00187-7

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. In G. Varoquaux, T. Vaught, & J. Millman (Eds.), *Proceedings of the 7th Python in Science Conference* (pp. 11–15). https://doi.org/10.25080/TCWV9851

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hoffmann, M., Kotzur, L., & Stolten, D. (2022). The Pareto-optimal temporal aggregation of energy system models. *Applied Energy*, *315*, 119029. https://doi.org/10.1016/j.apenergy.2022.119029

Hoffmann, M., Kotzur, L., Stolten, D., & Robinius, M. (2020). A review on time series aggregation methods for energy system models. *Energies*, *13*(3), 641. https://doi.org/10.3390/en13030641

Hörsch, J., Hofmann, F., Schlachtberger, D., & Brown, T. (2018). PyPSA-Eur: An open optimisation model of the European transmission system. *Energy Strategy Reviews*, *22*, 207–215. https://doi.org/10.1016/j.esr.2018.08.012

Klütz, T., Knosala, K., Behrens, J., Maier, R., Hoffmann, M., Pflugradt, N., & Stolten, D. (2025). ETHOS.FINE: A framework for integrated energy system assessment. *Journal of Open Source Software*, *10*(105), 6274. https://doi.org/10.21105/joss.06274

Kotzur, L., Nolting, L., Hoffmann, M., Groß, T., Smolenko, A., Priesmann, J., Büsing, H., Beer, R., Kullmann, F., Singh, B., Praktiknjo, A., Stolten, D., & Robinius, M. (2021). A modeler's guide to handle complexity in energy systems optimization. *Advances in Applied Energy*, *4*, 100063. https://doi.org/10.1016/j.adapen.2021.100063

Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, *28*(2), 129–137. https://doi.org/10.1109/TIT.1982.1056489

McInnes, L., Healy, J., & Astels, S. (2017). hdbscan: Hierarchical density based clustering. *Journal of Open Source Software*, *2*(11), 205. https://doi.org/10.21105/joss.00205

McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. https://doi.org/10.25080/Majora-92bf1922-00a

Patil, S., Kotzur, L., & Stolten, D. (2022). Advanced spatial and technological aggregation scheme for energy system models. *Energies*, *15*(24), 9517. https://doi.org/10.3390/en15249517

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel,

DRAFT

M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Plotly Technologies Inc. (2015). *Collaborative data science*. Plotly Technologies Inc. https://plot.ly

Schubert, E., & Lenssen, L. (2022). Fast k-medoids clustering in Rust and Python. *Journal of Open Source Software*, *7*(75), 4183. https://doi.org/10.21105/joss.04183

Schubert, E., & Rousseeuw, P. J. (2021). Fast and eager k-medoids clustering: O(k) runtime improvement of the PAM, CLARA, and CLARANS algorithms. *Information Systems*, *101*, 101804. https://doi.org/10.1016/j.is.2021.101804

Stöckl, B., Anarmo, M., Wogrin, S., & Werner, Y. (2026). Voltage-aware grid aggregation: Expanding the European high-voltage network. *2026 22nd International Conference on the European Energy Market (EEM)*, 1–6. https://doi.org/10.1109/EEM68581.2026.11589833

Stöckl, B., Werner, Y., & Wogrin, S. (2025). Congestion-sensitive grid aggregation for DC optimal power flow. *2025 IEEE Kiel PowerTech*, 1–7. https://doi.org/10.1109/powertech59965.2025.11180585

Teichgraeber, H., & Brandt, A. R. (2022). Time-series aggregation for the optimization of energy systems: Goals, challenges, approaches, and opportunities. *Renewable and Sustainable Energy Reviews*, *157*, 111984. https://doi.org/10.1016/j.rser.2021.111984

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … Vázquez-Baeza, Y. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Ward, J. H., Jr. (1963). Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, *58*(301), 236–244. https://doi.org/10.1080/01621459.1963.10500845

Wogrin, S. (2023). Time series aggregation for optimization: One-size-fits-all? *IEEE Transactions on Smart Grid*, *14*(3), 2489–2492. https://doi.org/10.1109/TSG.2023.3242467

Wood, A. J., Wollenberg, B. F., & Sheblé, G. B. (2013). *Power generation, operation, and control* (3rd ed.). John Wiley & Sons. ISBN: 978-0-471-79055-6

Xiong, B., Fioriti, D., Neumann, F., Riepin, I., & Brown, T. (2025). Modelling the high-voltage grid using open data for Europe and beyond. *Scientific Data*, *12*, 277. https://doi.org/10.1038/s41597-025-04550-7

Xiong, B., Fioriti, D., Neumann, F., Riepin, I., & Brown, T. (2026). *Prebuilt electricity network for PyPSA-Eur based on OpenStreetMap data*. Zenodo. https://doi.org/10.5281/ZENODO.18619025

DRAFT